DOCUMENTACIÓN DEL PROYECTO

Sistema de Detección de Procesos Sospechosos

Alumno: Osvaldo Emmanuel Lara Loera

Materia: Ciberseguridad

Fecha: Abril 04 del 2026

1. Introducción

Actualmente, la seguridad informática es un tema muy importante, ya que muchas amenazas pueden afectar el funcionamiento de una computadora sin que el usuario se dé cuenta. En muchos casos, los virus, scripts maliciosos o programas sospechosos se ejecutan como procesos dentro del sistema operativo, aparentando ser archivos o tareas normales.

Con base en esto, en este proyecto se desarrolló un sistema en **Python** que tiene como propósito **monitorear los procesos activos del sistema** y detectar aquellos que presenten características sospechosas. El programa analiza ciertos aspectos de cada proceso, como su nombre, la ruta desde donde se ejecuta y los comandos que utiliza, para determinar si puede representar un riesgo.

Este proyecto fue diseñado como una herramienta básica de detección, enfocada en aplicar conceptos de **ciberseguridad defensiva**, monitoreo de procesos y análisis de comportamiento.

2. Objetivo General

Desarrollar un programa capaz de **detectar procesos sospechosos en ejecución** dentro del sistema operativo, utilizando reglas básicas de análisis y generando un reporte automático de seguridad.

3. Objetivos Específicos

- Monitorear los procesos activos del sistema en tiempo real.
- Detectar procesos nuevos que aparezcan durante la ejecución.
- Analizar nombres de procesos y comandos utilizados.
- Identificar palabras clave relacionadas con software malicioso.
- Revisar si un proceso se ejecuta desde rutas sospechosas.
- Generar un reporte con la información de los procesos detectados.
- Evitar que el propio programa se detecte a sí mismo.
- Reducir la repetición de alertas innecesarias.

4. Descripción del Proyecto

El proyecto consiste en un programa hecho en **Python** que revisa constantemente los procesos que se están ejecutando en una computadora. Su función principal es **detectar comportamientos que puedan ser considerados sospechosos**, utilizando una lógica basada en puntajes.

Cada proceso es analizado según ciertas condiciones, por ejemplo:

- Si es un proceso nuevo.
- Si contiene palabras relacionadas con malware.
- Si se ejecuta desde carpetas como **Temp**, **AppData** o **Roaming**.
- Si utiliza programas como **cmd**, **powershell** o **python** para ejecutar scripts.

Cuando un proceso cumple con varias de estas condiciones, el sistema le asigna un **puntaje de riesgo**. Si el puntaje es suficientemente alto, el proceso es marcado como sospechoso y se registra en un archivo de reporte.

5. Herramientas Utilizadas

Para el desarrollo de este proyecto se utilizaron las siguientes herramientas y librerías:

Herramienta	Función

Python 3	Lenguaje de programación utilizado
psutil	Obtener información de procesos activos
os	Obtener rutas y datos del sistema
time	Controlar tiempos de ejecución
datetime	Registrar fecha y hora en el reporte

6. Funcionamiento General del Sistema

El sistema funciona de manera continua mientras se esté ejecutando. Cada cierto tiempo realiza un nuevo análisis de los procesos del sistema.

Proceso general de funcionamiento

1. El programa inicia el monitoreo.
2. Obtiene la lista de procesos activos.
3. Analiza cada proceso uno por uno.
4. Calcula un puntaje de riesgo.
5. Si el proceso es sospechoso, lo registra en el reporte.
6. Espera 2 segundos.
7. Vuelve a repetir el proceso.

De esta forma, el sistema trabaja como un **monitor básico de seguridad**.

7. Lógica de Detección

El programa utiliza una lógica sencilla pero funcional para decidir si un proceso puede ser considerado sospechoso.

7.1 Palabras sospechosas

Se definió una lista de palabras que suelen estar relacionadas con malware, scripts de prueba o herramientas maliciosas.

Palabras utilizadas

- virus
- malware
- miner
- hack
- test
- temp
- payload

Si alguna de estas palabras aparece en el nombre del proceso o en el comando con el que fue ejecutado, el sistema aumenta el nivel de sospecha.

7.2 Procesos nuevos

Cuando el programa detecta un proceso que no había visto antes, le asigna un puntaje inicial, ya que puede tratarse de una actividad reciente o inesperada.

7.3 Uso de comandos o shells

El programa presta especial atención a procesos como:

- `cmd.exe`
- `powershell.exe`
- `py.exe`

Esto se debe a que muchas veces estos programas son utilizados para ejecutar scripts o comandos de forma indirecta.

7.4 Rutas sospechosas

También se revisa si un proceso se está ejecutando desde rutas como:

- `AppData`
- `Temp`

- `Roaming`

Estas ubicaciones pueden ser utilizadas por programas maliciosos para esconderse dentro del sistema.

7.5 Ejecución de scripts Python

El sistema detecta cuando un proceso está ejecutando un archivo con extensión `.py`.

Si el nombre del script contiene alguna palabra sospechosa, se le asigna un puntaje mayor.

8. Sistema de Puntaje (Score)

Para decidir si un proceso es sospechoso, el programa utiliza un **sistema de puntaje**. Cada comportamiento detectado suma cierta cantidad de puntos.

Tabla de puntajes

Criterio detectado	Puntaje
Proceso nuevo	+1
Shell con comando sospechoso	+5
Shell con comando largo	+3
Shell desde ruta sospechosa	+4
Ejecución en ruta sospechosa	+3
Script sospechoso	+6
Script normal en ejecución	+2
Python ejecutando script	+2

Clasificación del riesgo

Puntaje total	Nivel de riesgo

6 a 7	**Medio**
8 o más	**Alto**

Solo se reportan los procesos que obtienen un puntaje de **6 o más**.

9. Generación del Reporte

El sistema guarda todas las alertas en un archivo llamado:

`reporte_seguridad.txt`

En este archivo se almacena la información de cada proceso sospechoso detectado.

Información guardada

- Fecha y hora
- PID del proceso
- Nombre del proceso
- Nivel de riesgo
- Puntaje obtenido
- Ruta del ejecutable
- Comando de ejecución
- Motivos por los que fue detectado

Ejemplo de alerta generada

```
ALERTA DETECTADA
PID: 4520
Proceso: python.exe
Riesgo: ALTO
Score: 9

Ruta:
c:\users\usuario\appdata\local\programs\python\python.exe

Comando:
python malware_test.py
```

```
Motivos:
 - Proceso nuevo
 - Script sospechoso: malware_test.py
 - Python ejecutando script
```

10. Estructura del Código

El programa está organizado en tres funciones principales:

10.1 Función `analizar_procesos()`

Función

Esta parte del programa se encarga de revisar todos los procesos activos y decidir cuáles son sospechosos.

Lo que hace

- Obtiene información de cada proceso
- Analiza nombre, ruta y comandos
- Detecta scripts o palabras sospechosas
- Calcula el puntaje de riesgo
- Guarda los procesos detectados en una lista

10.2 Función `generar_reporte()`

Función

Guarda la información de los procesos sospechosos en el archivo de reporte.

Lo que hace

- Registra la fecha y hora
- Escribe la información del proceso
- Clasifica el nivel de riesgo

- Muestra cuántos procesos fueron detectados

10.3 Función `monitoreo()`

Función

Mantiene el programa funcionando continuamente.

Lo que hace

- Inicia el monitoreo
- Ejecuta el análisis de procesos
- Genera el reporte
- Espera 2 segundos y vuelve a repetir

11. Medidas de Control Implementadas

11.1 Evitar auto-detección

El sistema fue programado para no detectarse a sí mismo, ya que de lo contrario podría generar alertas falsas sobre su propia ejecución.

Para esto:

- ignora su propio **PID**
- ignora su propia **ruta de ejecución**

11.2 Sistema de cooldown

También se implementó un tiempo de espera de **10 segundos** antes de volver a reportar el mismo proceso sospechoso.

Esto evita que el archivo de reporte se llene con la misma alerta repetida muchas veces.

12. Pruebas Realizadas

Para comprobar el funcionamiento del sistema, se realizaron pruebas con procesos simulados y scripts de prueba.

Pruebas aplicadas

- Ejecución de archivos llamados:
  - `virus.py`
  - `malware_test.py`
  - `payload.py`
- Ejecución de scripts desde carpetas temporales
- Uso de `cmd` y `powershell`
- Ejecución de scripts normales para comparar resultados

Resultado

Durante las pruebas, el sistema logró detectar correctamente procesos que contenían características sospechosas, registrando la información en el archivo de reporte.

Esto demuestra que el programa funciona correctamente como un sistema básico de monitoreo y detección.

13. Ventajas del Proyecto

- Es fácil de entender y de usar.
- Permite monitorear procesos en tiempo real.
- Aplica conceptos reales de ciberseguridad.
- Genera evidencia escrita de las alertas detectadas.
- Puede servir como base para desarrollar un sistema más avanzado.

14. Limitaciones del Proyecto

Aunque el sistema funciona correctamente, también tiene algunas limitaciones:

- No detecta malware real avanzado.
- No analiza el consumo de CPU o memoria.
- No revisa conexiones de red.
- No bloquea ni elimina procesos automáticamente.
- Puede detectar algunos procesos normales como sospechosos.

Estas limitaciones son normales porque se trata de una **primera versión funcional del proyecto**.

15. Posibles Mejoras

Para mejorar este sistema en una futura versión, se podrían agregar las siguientes funciones:

- Detección de procesos con alto consumo de **CPU**
- Detección de procesos con alto uso de **RAM**
- Monitoreo de actividad de **red**
- Lista blanca de procesos confiables
- Opción para cerrar procesos sospechosos
- Interfaz gráfica
- Exportación del reporte en otros formatos

16 MARCO TEÓRICO

Monitoreo de procesos en ciberseguridad

El monitoreo de procesos consiste en observar continuamente los programas que se están ejecutando dentro de un sistema operativo. Cada aplicación o tarea activa se representa como un proceso, el cual puede consumir recursos como CPU, memoria o acceso a archivos.

En el ámbito de la ciberseguridad, este monitoreo es importante porque muchos programas maliciosos se ejecutan como procesos normales, intentando pasar desapercibidos. Por ello, analizar su comportamiento permite detectar actividades sospechosas sin necesidad de conocer previamente el virus.

Detección por comportamiento vs detección por firma

Existen dos enfoques principales para detectar amenazas:

Detección por firma

Se basa en comparar archivos o procesos con una base de datos de virus conocidos.
Si coincide con una firma registrada, se considera malware.

Desventaja:
No detecta amenazas nuevas.

Detección por comportamiento (heurística)

Analiza lo que hace un proceso, en lugar de cómo se llama.

Ejemplos:

- Alto consumo de CPU
- Ejecución desde carpetas sospechosas
- Uso de comandos extraños

Ventaja:
Puede detectar amenazas nuevas o desconocidas.

En este proyecto:

Se utiliza **detección por comportamiento**, ya que el sistema analiza acciones sospechosas y asigna un puntaje de riesgo.

17. EDR (Endpoint Detection and Response)

Un sistema EDR (Endpoint Detection and Response) es una herramienta de seguridad que monitorea dispositivos como computadoras en tiempo real para detectar, analizar y responder a amenazas.

Los sistemas EDR permiten:

- Identificar procesos sospechosos
- Analizar comportamiento del sistema
- Registrar eventos de seguridad
- Generar alertas

Relación con el proyecto

El sistema desarrollado en este proyecto funciona como una **versión básica de un EDR**, ya que:

- Monitorea procesos en tiempo real
- Detecta comportamientos sospechosos
- Genera reportes de eventos

Framework MITRE ATT&CK

El framework MITRE ATT&CK es una base de conocimiento utilizada en ciberseguridad que clasifica las técnicas usadas por atacantes.

Algunas técnicas relacionadas con este proyecto son:

T1057 – Process Discovery

Consiste en obtener información sobre los procesos que se están ejecutando en el sistema.

Relación:
El programa analiza todos los procesos activos usando `psutil`.

T1059 – Command and Scripting Interpreter

Se refiere al uso de herramientas como:

- cmd
- PowerShell
- Python

para ejecutar comandos o scripts maliciosos.

Relación:
El sistema detecta el uso de estos programas junto con comandos sospechosos.

Indicadores de Compromiso (IoC)

Los IoC (Indicators of Compromise) son señales que indican que un sistema puede estar comprometido.

Ejemplos:

- Archivos con nombres sospechosos
- Procesos ejecutándose en rutas inusuales
- Uso excesivo de recursos

En el proyecto se usan IoC como:

- Palabras clave (virus, malware, etc.)
- Ejecución en carpetas como AppData o Temp
- Alto consumo de CPU

Falsos positivos

Un falso positivo ocurre cuando un sistema identifica un proceso legítimo como si fuera una amenaza.

Esto puede pasar porque:

- Algunos programas normales usan mucha CPU
- Scripts legítimos pueden parecer sospechosos

Solución aplicada en el proyecto

Se implementó un sistema de:

- **Puntaje (score)** → reduce decisiones extremas
- **Cooldown** → evita repetir alertas constantemente

Configuración externa en sistemas de seguridad

En herramientas reales de ciberseguridad, es importante que los parámetros no estén fijos en el código, sino que puedan modificarse fácilmente.

Por eso se utilizan archivos externos como:

- JSON
- YAML
- .env

Aplicación en el proyecto

Se implementó un archivo `config.json` que permite modificar:

- Umbral de CPU
- Intervalo de escaneo
- Lista de palabras sospechosas

Esto mejora:

- Flexibilidad
- Escalabilidad
- Mantenimiento del sistema

Uso de CPU como indicador de amenaza

El consumo elevado de CPU puede ser una señal de actividad maliciosa, especialmente en casos como:

- Minería de criptomonedas
- Procesos en bucle infinito
- Ejecución de scripts pesados

Aplicación en el proyecto

El sistema detecta procesos cuyo uso de CPU supera un umbral definido, considerándolo como comportamiento sospechoso.

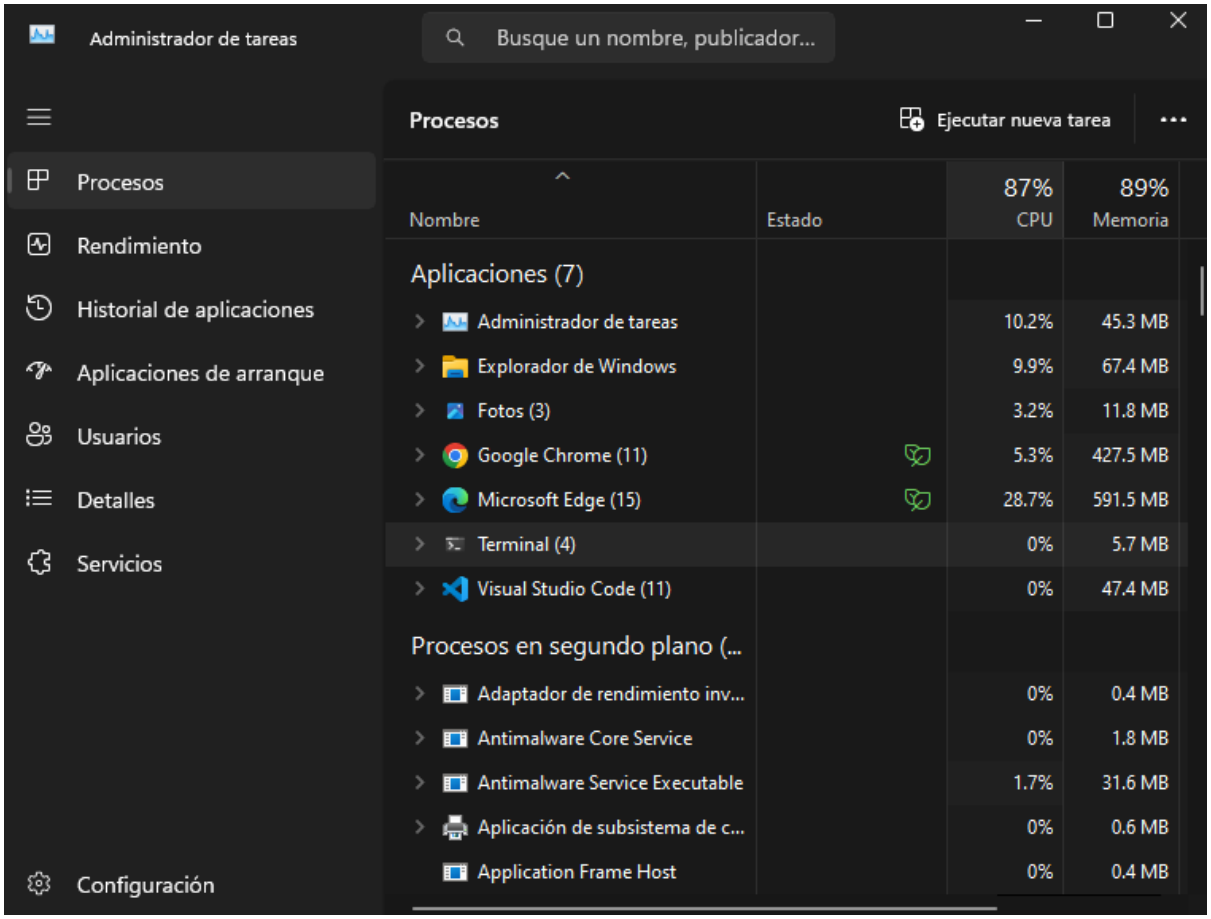
CONCLUSIÓN DEL MARCO TEÓRICO

El presente proyecto aplica conceptos fundamentales de la ciberseguridad moderna, como la detección basada en comportamiento, el monitoreo continuo de procesos y el uso de indicadores de compromiso.

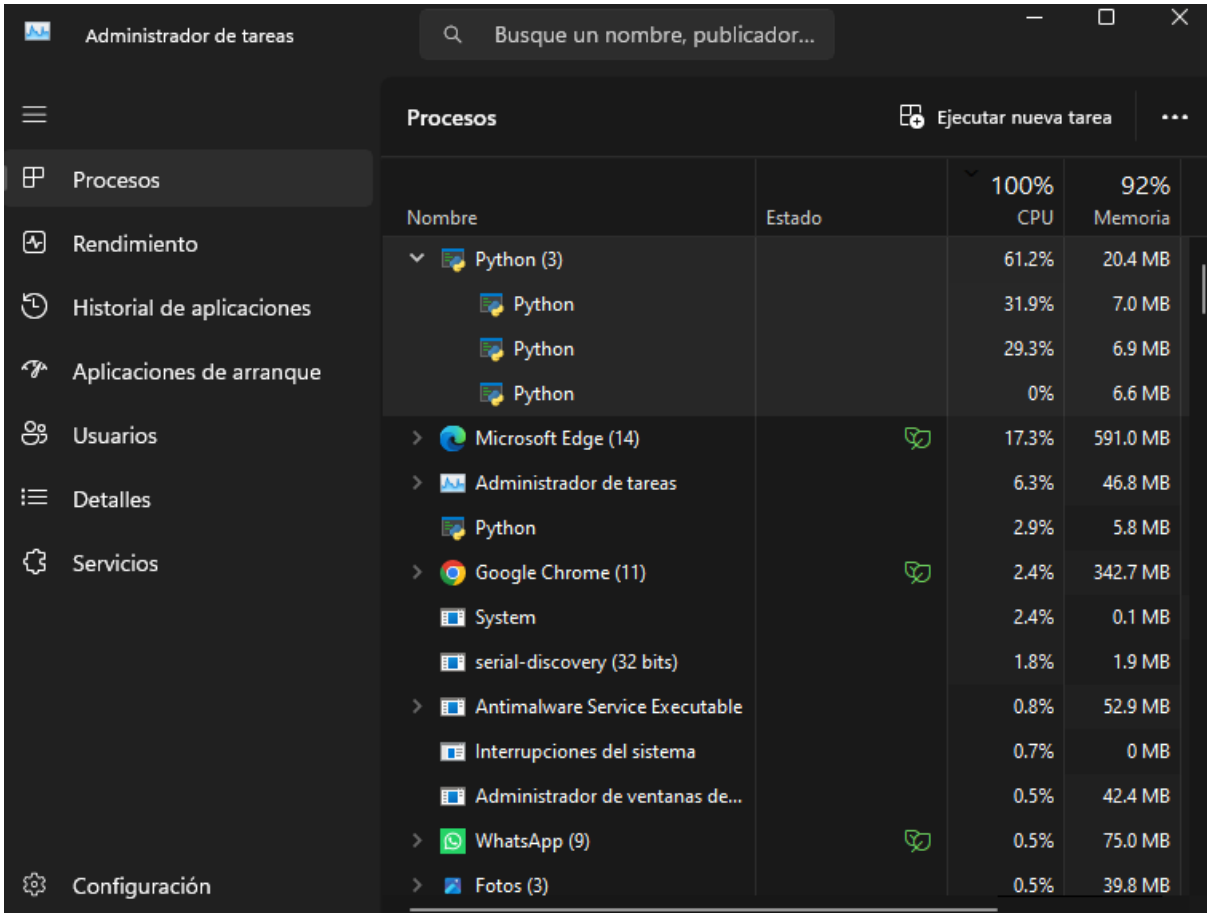
Además, incorpora prácticas utilizadas en herramientas reales, como la configuración externa y el análisis heurístico, lo cual permite identificar amenazas potenciales incluso sin contar con firmas específicas de malware.

EVIDENCIAS

CAPTURA DEL CPU ANTES DE EJECUTAR UN SCRIP DE CONSUMO DE CPU

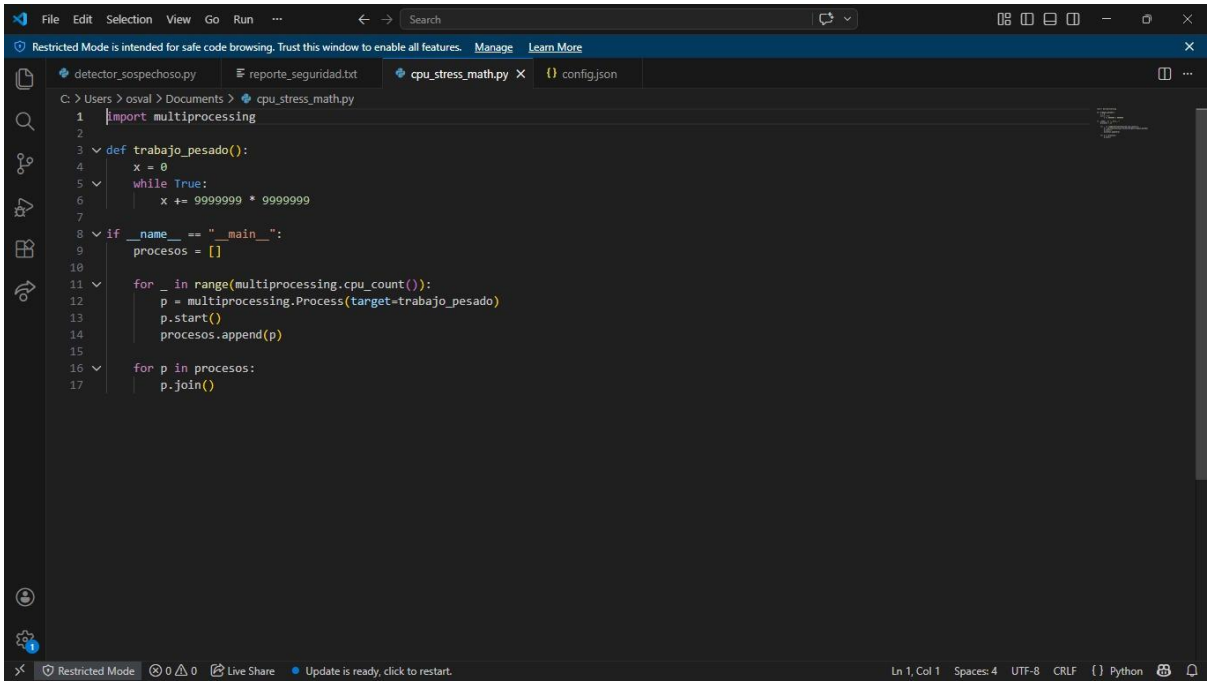


CAPTURA DESPUES DE LA EJECUCION DEL SCRIP



SCRIP QUE SE UTILIZO

```
PS C:\Users\osval\Documents> py cpu_stress_math.py
```



PROGRAMA CORRIENDO

```
Windows PowerShell   Windows PowerShell
  File "C:\Users\osval\Desktop\CIBERCEGURIDAD\detector_sospechoso.py", line 10, in <module>
    with open("config.json", "r") as f:
         ~~~~^^^^^^^^^^^^^^^^^^^^^^^^
PermissionError: [Errno 13] Permission denied: 'config.json'
PS C:\Users\osval\Desktop\CIBERCEGURIDAD> py detector_sospechoso.py
[INFO] Monitoreo iniciado...
[ Escaneo | Detectados: 1
[ Escaneo | Detectados: 0
[ Escaneo | Detectados: 0
[ Escaneo | Detectados: 0
[ Escaneo | Detectados: 0
[ Escaneo | Detectados: 0
[ Escaneo | Detectados: 0
[ Escaneo | Detectados: 0
[ Escaneo | Detectados: 4
[ Escaneo | Detectados: 3
[ Escaneo | Detectados: 3
[ Escaneo | Detectados: 2
[ Escaneo | Detectados: 0
[ Escaneo | Detectados: 0
[ Escaneo | Detectados: 0
[ Escaneo | Detectados: 0
[ Escaneo | Detectados: 0
[ Escaneo | Detectados: 0
[ Escaneo | Detectados: 0
[ Escaneo | Detectados: 0
[ Escaneo | Detectados: 2
[ Escaneo | Detectados: 2
[ Escaneo | Detectados: 2
[ Escaneo | Detectados: 2
```

ARCHIVO DE REPORTE

```
2832  Ruta:
2833  c:\users\osval\appdata\local\python\pythoncore-3.14-64\python.exe
2834
2835  Comando:
2836  c:\users\osval\appdata\local\python\pythoncore-3.14-64\python.exe -c from multiprocessing.spawn import spawn_main; spawn_main(parent_pid=15520,
2837
2838  Motivos:
2839   - Alto consumo de CPU: 101.9%
2840   - Proceso nuevo
2841   - Ejecución en ruta sospechosa
2842  ------------------------
2843
2844  ALERTA DETECTADA
2845  PID: 12572
2846  Proceso: python.exe
2847  Riesgo: ALTO
2848  Score: 9
2849  CPU: 74.9%
2850
2851  Ruta:
2852  c:\users\osval\appdata\local\python\pythoncore-3.14-64\python.exe
2853
2854  Comando:
2855  c:\users\osval\appdata\local\python\pythoncore-3.14-64\python.exe -c from multiprocessing.spawn import spawn_main; spawn_main(parent_pid=15520,
2856
2857  Motivos:
2858   - Alto consumo de CPU: 74.9%
2859   - Proceso nuevo
2860   - Ejecución en ruta sospechosa
2861  ------------------------
2862
2863  ALERTA DETECTADA
2864  PID: 15520
```

18. Conclusión

En conclusión, este proyecto permitió desarrollar un sistema básico de **detección de procesos sospechosos** utilizando Python y principios simples de monitoreo en ciberseguridad.

El programa analiza diferentes características de los procesos activos y, mediante una lógica de puntaje, puede identificar aquellos que presenten comportamientos fuera de

lo normal. Además, genera un reporte que permite guardar evidencia de los eventos detectados.

Aunque todavía se puede mejorar mucho, este proyecto cumple correctamente con su objetivo principal y representa una base sólida para seguir desarrollando herramientas de seguridad más completas.